

Proyecto Integrador I
Departamento de Computación y Sistemas Inteligentes
Universidad IcesiPlan de Pruebas

Proyecto: Buro - Consultorio Jurídico ICESI

Aplicación: Sistema de Gestión Buro (Subsistema B: Gestión de Casos)

Equipo de Desarrollo No.T4

Integrantes:

- Stevan Andrade Becerra - A00408087
- Juan José Gordillo - A00407891
- Harold Steven Arias - A00408095
- Samuel Andrés Bonilla - A00400101
- Juan Esteban Guerrero - A00404580

Versión del Documento: 1.0.1

Este plan de pruebas define la estrategia, el alcance y los recursos necesarios para validar el correcto funcionamiento del Subsistema B (Gestión de Casos) de la aplicación Buro. En el contexto de la asignatura Proyecto Integrador I, el enfoque será garantizar la calidad de los flujos críticos del ciclo de vida de un caso jurídico (registro, asignación, bitácora y cierre) dentro de un entorno de desarrollo y pruebas local, asegurando el cumplimiento de los requerimientos funcionales establecidos en el SRS

Se realizarán pruebas sobre las funcionalidades definidas en el Subsistema B (Gestión de Casos):

- **CRUD de Casos (RF6, RF12):** Creación de casos, carga de documentos, actualización de estados y cierre del caso
- **Reparto y Asignación (RF7):** Validación del algoritmo de asignación automática de casos a estudiantes y la reasignación manual
- **Trazabilidad (RF8, RF9):** Registro cronológico correcto en la bitácora de actuaciones (*Actuacion*) por cada acción realizada
- **Alertas y Disciplina (RF10, RF13):** Generación de notificaciones por cambio de estado y registro de memos disciplinarios (*MemoDisciplinario*)
- **API y Backend:** Pruebas de integración sobre los *endpoints* de Django Rest Framework gestionados por *CasoViewSet* y validación de *CasoSerializer*
- **Seguridad (RF46):** Validación de la clase *PermisoAccesoCaso* para garantizar que solo el estudiante asignado o el profesor puedan acceder a los detalles confidenciales

1.1.2 Fuera del alcance

- Pruebas de estrés, carga o rendimiento masivo (conurrencia de miles de usuarios)
- Pruebas de penetración (Pentesting) o auditorías avanzadas de ciberseguridad

- Pruebas detalladas sobre los módulos de Gestión de Usuarios, Citas o Académico, excepto en los puntos donde interactúen obligatoriamente con la creación de un caso

1.2 Objetivo de Calidad

Garantizar que el 100% de los flujos principales del módulo de Gestión de Casos operen de manera estable y correcta. El sistema debe permitir el ciclo completo de un caso sin presentar caídas del servidor (Error 500), garantizando la integridad de la base de datos relacional y el respeto absoluto a los permisos de acceso de los usuarios.

Dado el tamaño del equipo de desarrollo estudiantil, los roles serán asumidos de manera colaborativa y rotativa para evitar que un desarrollador pruebe su propio código:

- **Analista de Pruebas (QA Tester):** Encargado de diseñar los casos de prueba, ejecutar las peticiones en la API y realizar las pruebas manuales en la interfaz. Documenta los errores.
- **Desarrollador (Backend/Frontend):** Encargado de revisar los reportes de *bugs*, depurar el código en Django/React y subir las correcciones al repositorio.
- **Líder de Calidad:** Supervisa que se cumpla el cronograma de pruebas y evalúa si el módulo está listo para la presentación final.

2. Metodología de pruebas 2.1 Tipos y Niveles de pruebas

Considerando las restricciones de tiempo y alcance del proyecto, se ejecutarán los siguientes niveles de pruebas:

- **Pruebas de Integración (API):** Se utilizará Postman para enviar peticiones HTTP (GET, POST, PUT, DELETE) a los *endpoints* del *CasoViewSet*. Se verificará que el sistema responda con los códigos de estado HTTP correctos (ej. 200 OK, 201 Created, 403 Forbidden) y que los datos se guarden bien en la base de datos.
- **Pruebas Funcionales (Caja Negra):** Pruebas manuales simulando el comportamiento de los roles de "Secretaria", "Estudiante" y "Profesor" a través de la interfaz de usuario, verificando que los requerimientos de negocio se cumplan de manera visual y operativa.

2.2 Clasificación de errores

Los defectos encontrados se registrarán y clasificarán bajo los siguientes niveles de severidad:

- **Crítico:** El sistema colapsa (*Crash/Error 500*) o hay un bloqueo total que impide registrar un caso, realizar el reparto o iniciar sesión
- **Alto:** Una funcionalidad vital falla, pero existe una ruta alternativa; o se detecta una brecha en los permisos (ej. un estudiante ve un caso que no es suyo)
- **Medio:** Errores de validación de datos (el sistema permite fechas incongruentes) o las notificaciones no se generan en el momento adecuado
- **Bajo:** Defectos cosméticos en la interfaz, errores ortográficos o desajustes visuales

que no impiden el flujo de trabajo

2.3 Seguimiento y control

El seguimiento del avance de las pruebas se realizará utilizando un tablero Kanban (Jira o Trello). Se utilizarán las siguientes métricas de seguimiento:

- **Densidad de Defectos:** Cantidad de *bugs* reportados clasificados por severidad.
- **Porcentaje de Ejecución:** Número de Casos de Prueba ejecutados vs. Casos de Prueba planificados.

2.3.1 Criterios de suspensión y requisitos de reanudación

- **Criterios de Suspensión:** La ejecución de pruebas sobre un flujo se detendrá inmediatamente si se detecta un error de severidad **Crítica** que bloquee el proceso (ejemplo: la base de datos rechaza las inserciones o el sistema de autenticación cae).
- **Requisitos de Reanudación:** Las pruebas se retomarán únicamente cuando el desarrollador asigne el *ticket* a estado "Resuelto" y los cambios se hayan integrado exitosamente en la rama principal del repositorio (*main* o *develop*)

2.3.2 Completitud de la prueba

La fase de pruebas del Subsistema B se considerará oficialmente completa cuando:

1. Se haya ejecutado el 100% de los casos de prueba definidos en la matriz
2. La métrica de defectos indique **cero (0)** errores de severidad Crítica o Alta en estado "Abierto"
3. Los Criterios de Aceptación de las Historias de Usuario principales del módulo se cumplan a cabalidad

2.5 Tareas del proyecto, estimación y cronograma

Fase / Tarea	Responsable	Estimación (Horas)
1. Diseño: Redacción de la matriz de casos de prueba (pasos, datos, resultados).	Stevan Andrade Becerra	4 horas ▾
2. Configuración: Creación de datos semilla (<i>Mocks</i>) en la BD local y colección en Postman.	Juan José Gordillo	2 horas ▾
3. Ejecución API: Prueba de <i>endpoints</i> de DRF mediante Postman.	Harold Steven Arias	5 horas ▾

4. Ejecución Funcional: Navegación manual por la plataforma simulando roles.	Samuel Andrés Bonilla	5 horas ▾
5. Reporte y Corrección: Registro de incidencias en el tablero y corrección de código.	Todo el equipo	8 horas ▾
6. Cierre (Regresión): Verificación final de los errores corregidos.	Juan Esteban Guerrero	3 horas ▾

3. Entregables de prueba

Al concluir el proceso de pruebas, el equipo generará los siguientes artefactos:

1. **Matriz de Casos de Prueba (Excel/PDF):** Documento detallado con el registro de cada prueba, indicando los pasos ejecutados y el resultado (Pasa/Falla).
2. **Colección de Postman (.json):** Archivo exportado con las peticiones configuradas listas para reproducir las pruebas de la API.
3. **Reporte de Incidencias:** Exportación o captura del tablero Kanban evidenciando los *bugs* encontrados, su clasificación y resolución.

4. Necesidades de recursos y ambientes

Para ejecutar las pruebas correctamente, se requiere:

- **Hardware:** Computadoras personales de los integrantes del equipo con capacidad de correr el servidor local (*localhost*).
- **Software:**
 - Entorno de ejecución: Python (Django Rest Framework) y entorno virtual configurado.
 - Base de datos local: PostgreSQL o SQLite (según configuración del proyecto).
 - Herramientas de pruebas: Postman (para API) y navegadores web actualizados (Chrome/Firefox).
 - Gestión: Tablero Kanban en Trello, GitHub Projects o Jira.
- **Personas:** Los estudiantes del equipo de desarrollo (Stevan, Juan José, Harold, Samuel y Juan Esteban), asumiendo los roles descritos en el apartado 1.3.